

Storage Wars

You added an index. Postgres ignored it. Here's why.

Issue #2

Fri, 02 Oct 2026

⌚ ~19 min read

Level 3/5

⚡ The short version

1. **A composite index is a phone book sorted by (surname, first name).** Column order decides which queries it can serve, and the planner skips any index its statistics say won't pay off. Part 1 covers composite and covering indexes, selectivity, and the usual reasons an index gets ignored.
2. **Pgpool-II patched seven CVEs on 1 October**, most of them in its watchdog/failover code. If Pgpool sits in front of your Postgres, upgrade this week.
3. **Open table formats keep spreading:** Cloudflare launched **Basin**, a serverless analytics platform built on Iceberg and R2, and **ClickHouse 26.9** added incremental materialized views that can feed Iceberg lakes.

Release radar

18.6

current **PostgreSQL** minor; **19** is still at **Beta 4**, with the RC due in early October

26.9

latest **ClickHouse** release (23 Sep), with 56 new features and 135 performance optimizations

41 days

until **PostgreSQL 14** end-of-life on 12 Nov 2026

Sources: postgresql.org, PostgreSQL 19 Beta 4, ClickHouse 26.9.

PART 1 Learn

🔍 Indexes in depth: composite, covering, selectivity, and when an index is ignored

FOUNDATIONS

~10 MIN

LEVEL 3

You add an index on `(customer_id, created_at)`, deploy, run `EXPLAIN`, and it still says `Seq Scan`. The index exists. It isn't broken. The planner looked at it and decided reading the whole table was cheaper, or the query is shaped so the index can't help. Yesterday we saw that **an index is a B-tree of pages pointing at rows**. Today: how the planner decides whether to use one, and how to design indexes it actually picks.

A COMPOSITE INDEX IS A SORTED PHONE BOOK

A multi-column B-tree index sorts its entries by the first column, then by the second within each first-column value, and so on. Use The Index, Luke has the best analogy: it's a **telephone directory sorted by surname, then first name**. Searching it by first name alone means reading every page. (Use The Index, Luke)

The PostgreSQL docs give the exact rule: **equality constraints on the leading columns, plus any inequality on the first column without an equality constraint**, limit the part of the index that gets scanned. Conditions on later columns are still checked against the index entries, but they don't narrow the range. (PostgreSQL docs)

Take an index on `orders (customer_id, status, created_at)`:

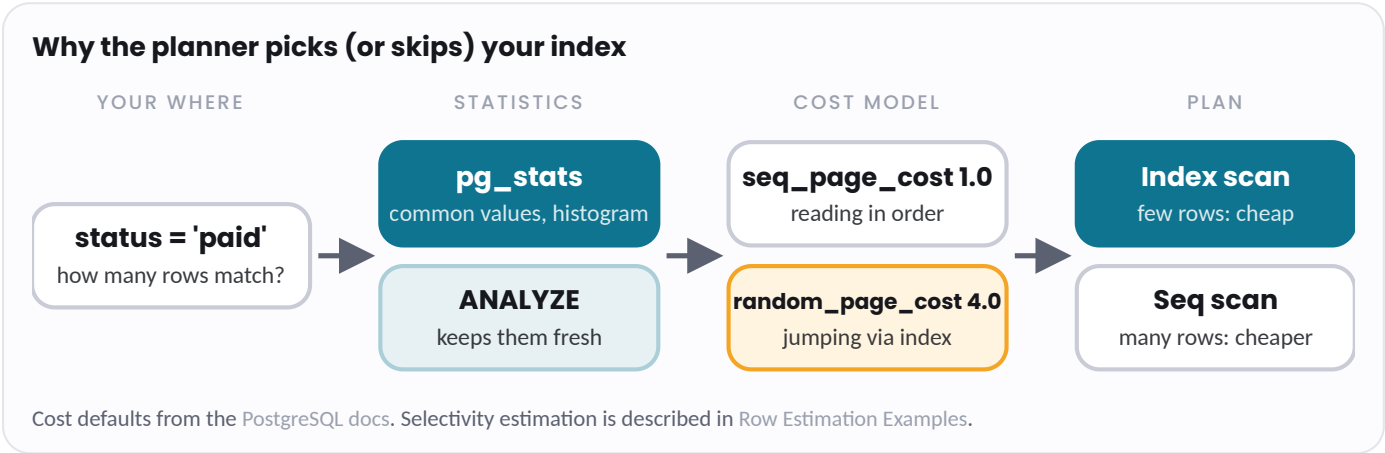
Query filter	Narrows the scan?	Why
<code>customer_id = 7</code>	✔ Yes	Leading column, equality
<code>customer_id = 7 AND status = 'paid'</code>	✔ Yes	Two leading equalities
<code>customer_id = 7 AND status = 'paid' AND created_at > now() - '7 days'</code>	✔ Fully	Equalities, then one range: the ideal shape
<code>customer_id = 7 AND created_at > ...</code>	⚠ Partly	Range on <code>created_at</code> can't narrow; <code>status</code> sits in between
<code>status = 'paid'</code>	✗ Usually not	No leading column (but see skip scan below)
<code>customer_id = 7 OR status = 'paid'</code>	✗ No	An OR across columns isn't a prefix

MySQL documents the same **leftmost-prefix rule** with an index on `(last_name, first_name)`: `WHERE first_name = 'John'` can't use it, and neither can `last_name = 'Jones' OR first_name = 'John'`. (MySQL docs)
That gives the working rule: **equality columns first, then the one range or sort column**.

What changed in PostgreSQL 18: skip scan. Postgres 18 can now use a multicolumn B-tree when there's no condition on the leading column, as long as later columns have useful conditions. (PostgreSQL 18 release notes) It does this by running repeated index searches, one per leading value. That only pays off **when the leading column has few distinct values**. With many distinct values the planner usually prefers a sequential scan. (PostgreSQL docs)
An index on `(region, customer_id)` with four regions can now serve `WHERE customer_id = 42`. An index on `(tenant_id, customer_id)` with 50,000 tenants probably still can't.

HOW THE PLANNER DECIDES

The planner doesn't ask "is there an index?" It asks "**which plan has the lowest estimated cost?**". That estimate starts with how many rows it expects a condition to match: the **selectivity**.



`ANALYZE` samples each column and stores its **most common values and their frequencies**, plus a **histogram** of the rest. For `status = 'paid'`, if `paid` is a common value the planner uses its recorded frequency directly. Otherwise it spreads the remaining frequency evenly over the other distinct values. Range conditions are estimated from the histogram buckets. (PostgreSQL docs) The level of detail is set by `default_statistics_target` (default **100**) and can be raised per column. (PostgreSQL docs)

Then it costs the options. By default a sequential page read costs **1.0** and a random one **4.0**. The docs note that real random reads are usually much worse than 4× sequential, but the default assumes most indexed reads hit cache. (PostgreSQL docs) Each matching row found through an index may mean a random page read. **If a condition matches a large share of the table, reading every page in order wins.** That's not a bug. It's the planner being right.

Two consequences people trip on:

- **Tiny test tables lie.** The docs: selecting 1 of 100 rows "will hardly be" an index candidate, because the 100 rows probably fit in a single page. **Test plans on realistic data volumes.** (PostgreSQL docs)
- **Stale statistics make bad plans.** "Always run ANALYZE first" is the docs' first rule for checking index usage. (PostgreSQL docs)

COVERING INDEXES: SKIP THE TABLE ENTIRELY

A normal index scan finds the row's address, then visits the heap page to fetch the columns. If **every column the query needs is in the index**, Postgres can do an **index-only scan** and skip the heap. B-tree indexes always support this. (PostgreSQL docs)

You can add columns purely to be carried along, without making them part of the search key, with `INCLUDE` :

```
-- Search on customer_id, carry total along for free reads
CREATE INDEX orders_cust_cov ON orders (customer_id) INCLUDE (total);

SELECT total FROM orders WHERE customer_id = 42;  -- index-only scan candidate
```

Key column or INCLUDE payload?

🔑 Key column: (customer_id, total)

- Sorted on, so it can narrow searches and serve ORDER BY
- Stored at **every level** of the tree
- Part of uniqueness if the index is UNIQUE

```
CREATE INDEX ON orders (customer_id,
total);
```

VS

📦 Payload: (customer_id) INCLUDE (total)

- Not searchable, just carried to the leaf
- **Truncated from upper levels**, so internal pages stay small
- UNIQUE applies to key columns only

```
CREATE INDEX ON orders (customer_id)
INCLUDE (total);
```

Source: PostgreSQL: index-only scans and covering indexes. Only B-tree, GiST and SP-GiST support INCLUDE.

There's a catch that only makes sense once you know MVCC. The index has no visibility information, so Postgres checks the **visibility map**, one bit per heap page that says "every row here is visible to everyone". If

the bit is set, it skips the heap. If not, it visits the heap anyway. The map is about four orders of magnitude smaller than the heap, which is why the trick is cheap, but it only works when **most pages are all-visible**, typically on slowly changing data. (PostgreSQL docs)

InnoDB gets some of this for free. As we saw yesterday, every secondary index carries the primary key, so an index on `(email)` already covers `SELECT id FROM users WHERE email = ...`. (MySQL docs)

FIVE REASONS YOUR INDEX IS BEING IGNORED

1. **Wrong column order.** The query filters on a non-leading column. Reorder, add an index, or check whether PG 18's skip scan applies.
2. **A function wraps the column.** `WHERE lower(email) = 'a@b.in'` can't use an index on `email`. It needs an **expression index** on `lower(email)`. (PostgreSQL docs)
3. **LIKE 'abc%' in a non-C locale.** The default operator class follows collation rules, so prefix matching can't use it. Create the index with `text_pattern_ops`, and keep a default index too if you also need `<` and `>`. (PostgreSQL docs)
4. **The condition isn't selective.** `WHERE status = 'paid'` matching 60% of rows should be a sequential scan.
5. **The statistics are stale or too coarse.** Run `ANALYZE`, or raise the statistics target on skewed columns.

To test whether an index is usable at all, the docs suggest temporarily setting `enable_seqscan = off` in your session. **If the plan still avoids the index, the query and the index don't match.** (PostgreSQL docs)

In the real world: GitLab caps indexes at 15 per table

GitLab's public database guide for its engineers treats every index as a cost: each write must update every index, and indexes take disk space. Before adding one, developers must ask whether the query can **re-use an existing index**, whether there's enough data for an index to beat scanning, and whether the maintenance cost is worth the faster query. **GitLab enforces a limit of 15 indexes per table.** It finds dead weight by looking for `idx_scan = 0` in `pg_stat_user_indexes` and tracks index scans over six months on GitLab.com. Its composite-index example matches today's table: an index on `(project_id, commit_sha)` serves `project_id` alone, but not `commit_sha` alone. On populated tables it requires `CREATE INDEX CONCURRENTLY`, which doesn't block normal traffic. (GitLab docs)

Gotcha: the index-only scan that isn't

You build a perfect covering index, `EXPLAIN` proudly says **Index Only Scan**, and it's still slow. Look for **Heap Fetches:** in `EXPLAIN ANALYZE`. A big number means the visibility map bits weren't set, so Postgres visited the heap for most rows anyway. That happens on tables with heavy recent writes, before `VACUUM` has caught up. The docs warn that payload columns also **duplicate table data and bloat the index**, so add them only when the table changes slowly enough for index-only scans to happen. (PostgreSQL docs) A covering index on a hot, constantly updated table can cost you on every write and save you nothing on reads.

Try it (15 minutes, Docker)

```
-- docker run -d --name pg -e POSTGRES_PASSWORD=pw postgres:18
-- docker exec -it pg psql -U postgres

CREATE TABLE orders (id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    region int, customer_id int, total numeric(10,2));
INSERT INTO orders (region, customer_id, total)
SELECT g % 4, (random()*100000)::int, (random()*500)::numeric(10,2)
FROM generate_series(1, 1000000) g;
CREATE INDEX orders_region_cust ON orders (region, customer_id);
ANALYZE orders;

-- 1. leading + second column, then 2. no leading column
EXPLAIN ANALYZE SELECT * FROM orders WHERE region = 2 AND customer_id = 4242;
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 4242;

-- 3. matches ~25% of rows: expect a seq scan
EXPLAIN ANALYZE SELECT * FROM orders WHERE region = 2;

-- 4. covering index; run the last query, UPDATE, then run it again
CREATE INDEX orders_cust_cov ON orders (customer_id) INCLUDE (total);
VACUUM orders;
EXPLAIN ANALYZE SELECT total FROM orders WHERE customer_id = 4242;
UPDATE orders SET total = total + 1 WHERE customer_id = 4242;
```

Compare the plan type and the `Buffers`: line for each query. In step 4, compare `Heap Fetches` before and after the `UPDATE`: the update clears the visibility bits on the pages it touched until the next `VACUUM`. Then try `SET enable_seqscan = off;` before query 3 and compare the timings. Most of the time, the planner was right.

WHY IT MATTERS TO YOU



Day-to-day impact



Interview value



Production risk



USE IT TOMORROW

1. **Design from the query, not the table.** Equality columns first, then the one range or `ORDER BY` column.
2. **Before adding an index, check whether an existing one already has your column as a prefix.**
3. **Use `INCLUDE` for hot, read-mostly lookups**, and check `Heap Fetches` to confirm it's working.
4. **Hunt for unused indexes** with `idx_scan = 0` in `pg_stat_user_indexes`, over a long enough window to cover month-end jobs.
5. **When the planner "ignores" your index, check selectivity and statistics before reaching for hints.**



Under the hood

- **Limits:** Postgres indexes can have up to **32 columns**, including `INCLUDE` columns (a compile-time setting). (PostgreSQL docs) MySQL allows **16**. (MySQL docs) The Postgres docs also warn that indexes with

more than three columns are "unlikely to be helpful" unless the workload is very stylized.

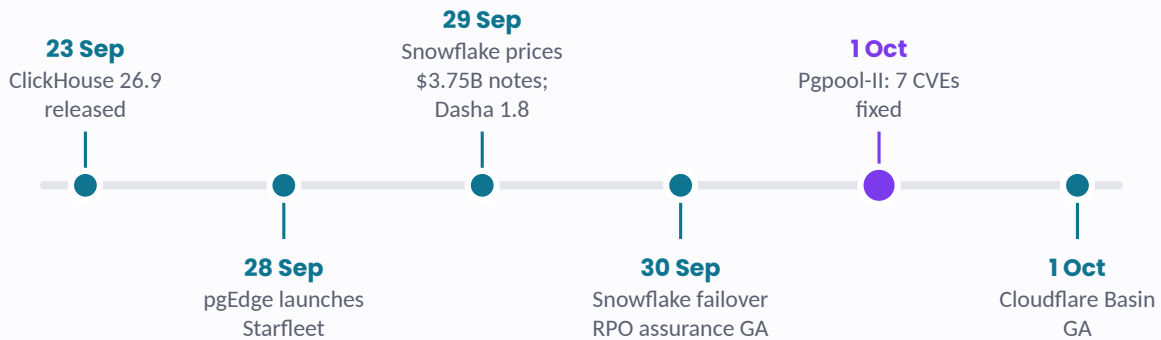
- **Expression indexes are computed on write.** The expression runs on every insert and non-HOT update, but not during an indexed search, because the result is already stored. [\(PostgreSQL docs\)](#)
- **Index-only scans need the base column.** The planner won't treat `(lower(email))` as covering `SELECT email`. The docs' workaround is to add the column as payload: `CREATE INDEX ON tab (f(x)) INCLUDE (x)`. [\(PostgreSQL docs\)](#)
- **MySQL invisible indexes** let you test dropping an index safely: `ALTER TABLE t ALTER INDEX idx INVISIBLE`. The optimizer stops using it, but it's still maintained (and still enforces uniqueness), so switching back is instant. Primary keys can't be made invisible. [\(MySQL docs\)](#)

GO DEEPER

- **Multicolumn Indexes** and **Index-Only Scans and Covering Indexes** [\(PostgreSQL docs\)](#). Short, precise, and they cover skip scan and the visibility map.
- **Multiple-Column Indexes** [\(MySQL docs\)](#). The leftmost-prefix rule with worked examples.
- **Concatenated Keys** [\(Use The Index, Luke\)](#). The best free chapter on column order.
- **Adding Database Indexes** [\(GitLab docs\)](#). A real team's checklist, from partial indexes to finding unused ones.

~10 min · A security patch for Postgres's most popular middleware, Iceberg everywhere, and Snowflake raises \$3.75 billion.

The week in data, 23 Sep → 1 Oct



Top story 1 · Pgpool-II patches seven security holes

SECURITY

UPGRADE NOW

What happened. On **1 October**, the Pgpool-II project released **4.7.3**, **4.6.8**, **4.5.13**, **4.4.18** and **4.3.21**, fixing **seven CVEs**. ([postgresql.org](https://www.postgresql.org)) Most sit in the **watchdog**, the component that coordinates several Pgpool nodes and decides which one leads:

- An **arbitrary 32-bit memory write** and a **buffer overflow** in watchdog message processing.
- **Stack corruption** from arbitrary-length data in failover messages.
- A **NULL pointer dereference** in watchdog inter-node authentication, and an **authentication-key bypass** that lets a node promote itself to leader.
- An **information leak** in the heartbeat receiver.
- A **certificate authentication bypass**: a NUL byte in an X.509 certificate's CN field.

Why it matters. Pgpool-II is a widely used connection pooler, load balancer and failover manager for Postgres. **Failover logic is the worst place for a bug**: an attacker who can talk to the watchdog could crash nodes or force a leader change, and the certificate bypass undermines client authentication. The announcement doesn't say whether a non-watchdog setup is fully safe, so don't assume it is.

👁️ What to do. Run Pgpool-II? **Upgrade to the patched release in your line**: each line from 4.3 to 4.7 got one. Until then, make sure watchdog and heartbeat ports are reachable only from the other Pgpool nodes. On a managed service? Check whether your provider bundles Pgpool. Details are in the [release notes](#).

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Production risk



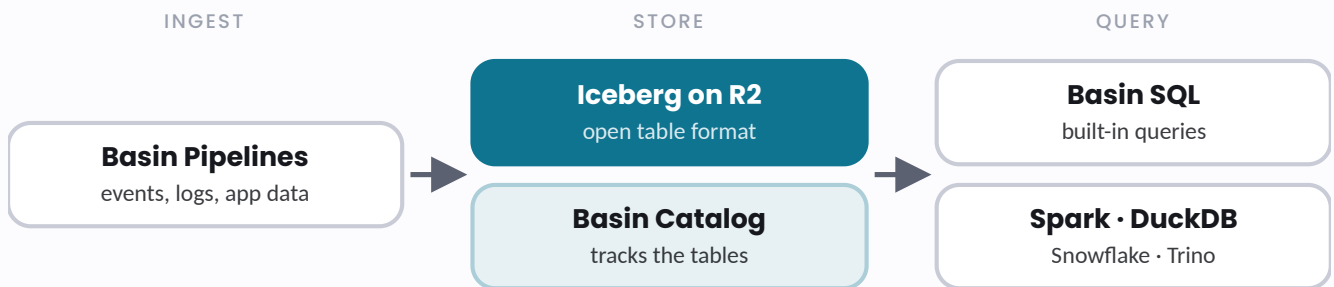
Top story 2 · Cloudflare Basin: an analytics platform made of open parts

LAKEHOUSE

GA

What happened. Cloudflare launched **Basin** on **1 October** as generally available: a serverless data platform to **ingest, store, catalog and query** analytical data. It has three named pieces, **Basin Pipelines** (ingestion), **Basin Catalog** and **Basin SQL**, and stores data as **Apache Iceberg** tables in **Cloudflare R2** object storage. Cloudflare says the same tables can be queried from **Spark**, **DuckDB**, **Snowflake** and **Trino**, with **no data transfer fees** between clouds and regions. Pricing wasn't in the announcement. (BigDATAwire)

Basin's shape: open format in the middle



Component names and engine list as stated in the launch announcement (BigDATAwire). Cost claims are Cloudflare's.

Why it matters. It's yesterday's theme again: **the open table format is the product, and engines plug into it.** Databricks shares foreign Iceberg tables, ClickHouse reads Iceberg in OneLake, and now a CDN company sells a lakehouse. One launch customer, Anomaly, says it replaced an S3 + Athena setup. For teams whose biggest analytics bill is **egress**, zero transfer fees are the real pitch. Treat the savings as vendor claims until you've priced your own workload.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Top story 3 · ClickHouse 26.9: incremental views and answers from metadata

CLICKHOUSE

RELEASE

What happened. ClickHouse 26.9 shipped on **23 September** with **56 new features**, **135 performance optimizations** and **464 bug fixes**. (ClickHouse blog) The highlights:

- **Incremental refreshable materialized views** (`APPEND INCREMENTAL`): process only newly committed rows instead of rescanning the source. ClickHouse pitches this for pushing append-only event streams out to **Iceberg** lakes.
- **min/max/count from statistics**: these queries can be answered from per-part metadata **without reading the column data**.
- `CREATE TOKEN` : time-limited, privilege-scoped credentials, so CI jobs and apps don't need the main password.
- **DISTINCT spills to disk** (`max_bytes_before_external_distinct`) instead of running out of memory.
- `LIMIT ... AFTER / UNTIL` clauses, and bracket syntax for JSON keys containing dots or spaces.

Why it matters. Answering from metadata is the same idea as today's **index-only scan**: the fastest read is one that never touches the data. Incremental views and short-lived tokens fix two common pain points of running ClickHouse in production: full rescans and long-lived shared credentials.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



⚡ Quick hits

❄️ **Snowflake failover gets an RPO target (GA, 30 Sep).**

`OPTIMIZED_REFRESH = TRUE` makes refresh time scale with your rate of change rather than object count, and `RPO_ASSURANCE = TRUE` runs continuous refreshes to keep secondaries in sync. Both are opt-in; existing groups stay on "Replication Classic". (Snowflake docs)

💰 **Snowflake raises \$3.75B in 0% convertible notes (29 Sep),**

split into 2029 and 2031 tranches. Proceeds pay for capped calls (~\$383.5M) and buying back ~\$262M of 2027 notes, plus general purposes. (Snowflake)

👁️ **Snowflake 10.35**

adds an `ACCOUNT_USAGE.STREAMS` view: one row per stream, with its source, mode and staleness. Handy for finding streams about to go stale. (Snowflake docs)

🔍 **Dasha 1.8 recommends indexes (29 Sep).**

The open-source Postgres fleet dashboard now reads `pg_stat_statements` across hosts and proposes missing B-tree indexes with column order and a ready `CREATE INDEX CONCURRENTLY`. It also adds `pg_stat_io` analysis and 17 schema checks. (postgresql.org)

🚀 **pgEdge Starfleet (28 Sep):**

a new Postgres cloud platform with copy-on-write branching on standard Postgres storage, an MCP server, on-prem and multi-region options. From \$25/month, per pgEdge. (postgresql.org)

📅 **Coming up.**

The PostgreSQL 19 RC (still due in early October), Postgres 14 end-of-life on 12 Nov, the DuckDB v2.0 release, and MongoDB's permanent CEO search.

📌 Revision card

- A composite index is sorted **column by column**: equality on leading columns plus **one range on the next** narrows the scan.
- Order columns as **equality first, then range/sort**. A non-leading filter usually can't use the index.
- **PG 18 skip scan** helps when the leading column has **few distinct values**.
- The planner picks by **estimated cost**: selectivity from `pg_stats` (common values + histogram) and `random_page_cost` **4.0** vs `seq_page_cost` **1.0**.
- **Low selectivity or tiny tables** → **seq scan**, correctly. Run `ANALYZE` before blaming the planner.
- Functions need **expression indexes**; `LIKE 'x%'` in non-C locales needs `text_pattern_ops`.
- **Covering = index-only scan**, via `INCLUDE`. It only pays off when the visibility map is set: watch **Heap Fetches**.
- **Pgpool-II: 7 CVEs fixed on 1 Oct**, so upgrade. **Basin and ClickHouse 26.9** push Iceberg further.

Storage Wars is researched and written by Claude every morning from official docs and release notes, project blogs and engineering write-ups. Vendor numbers are labelled as claims, and every item links to its source. Want more Kafka, less Mongo, or a different depth? Change `data/settings.yml` or just reply. 📧